



RETHINKING • HOW • WORK • WORKS

# **Agentic AI Engineering Internship , 2026**

## **WEEK 6**

### **AI Fundamentals & Agentic AI Foundations**

Monday 27 July – Friday 31 July 2026

*Giving agents a past, a world model, and the ability to grow smarter over time*

# Week Overview

A stateless agent forgets everything the moment a conversation ends. Week 6 fixes that. Interns build the full memory stack – episodic memory of past events, semantic memory of facts and preferences, procedural memory of learned behaviours, and structured knowledge graphs that let agents reason about relationships between entities, not just keyword matches.

By Friday, every intern's agent will remember users across sessions, learn from corrections, and traverse a structured knowledge graph to answer questions that retrieval alone cannot handle.

| Detail                 | Information                                                          |
|------------------------|----------------------------------------------------------------------|
| Week Dates             | Monday 27 July – Friday 31 July 2026                                 |
| Theme                  | Memory Systems & Knowledge Graphs                                    |
| Primary Stack          | Python · LangChain · LangGraph · ChromaDB · NetworkX · SQLite · Mem0 |
| Total Commitment       | 20 hours (4 hours/day · 5 days)                                      |
| Deliverable Due Friday | 1 Intermediate Project + 1 Production Project (choose one of each)   |

## Learning Objectives

- Understand and implement all four types of agent memory: working, episodic, semantic, procedural
- Build persistent memory stores using vector databases (ChromaDB) and relational stores (SQLite)
- Implement memory retrieval strategies: recency weighting, relevance scoring, importance decay
- Build knowledge graphs with NetworkX and query them with natural language
- Extract structured entities and relationships from unstructured text automatically
- Design memory compression and consolidation – handling the context window limit gracefully

- Build an agent that measurably improves over repeated interactions through memory

## The Four Memory Types

| Memory Type       | What It Stores                                                                | Implementation                                   | Example Use                                    |
|-------------------|-------------------------------------------------------------------------------|--------------------------------------------------|------------------------------------------------|
| Working Memory    | Current conversation context, active task state, intermediate results         | LangGraph state, in-context messages             | Tracking steps in a multi-turn task            |
| Episodic Memory   | Past events, conversation histories, interaction logs with timestamps         | SQLite with timestamps + ChromaDB semantic index | Remembering a user asked about X two weeks ago |
| Semantic Memory   | Facts, preferences, entity attributes, generalised knowledge                  | ChromaDB embeddings + key-value store            | Knowing a user prefers concise answers         |
| Procedural Memory | Learned skills, correction history, improved prompts, tool selection patterns | SQLite correction log + dynamic prompt builder   | Never making the same mistake twice            |

## Topics & Subtopics

| Topic                | Subtopics                                                                                                   |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| Memory Architecture  | Four memory types, memory lifecycle (encode → store → retrieve → forget), memory isolation per user/session |
| Storage Backends     | SQLite (episodic + procedural), ChromaDB (semantic), Redis (working cache), hybrid stores                   |
| Retrieval Strategies | Recency weighting, importance scoring, relevance-recency blending, memory compression triggers              |
| Knowledge Graphs     | Graph concepts (nodes, edges, properties), NetworkX construction, entity extraction, relationship typing    |
| Graph Querying       | Natural language to graph traversal, multi-hop reasoning, neighbourhood expansion, path finding             |
| Memory Consolidation | Summarising old episodes, importance-based forgetting, memory deduplication, profile building               |
| GraphRAG Pattern     | Combining vector retrieval (ChromaDB) with graph traversal (NetworkX) for richer context                    |

# Daily Breakdown

The week builds a full memory stack layer by layer. Do not skip ahead – each day's lab feeds directly into the next.

| Day        | Focus                                 | Core Learning                                                                                                                                                                                                           | Applied Practice                                                                                                                                                                                                  |
|------------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mon</b> | <b>Episodic &amp; Semantic Memory</b> | Study the four memory types. Read MemGPT paper. Build a basic episodic store: SQLite table logging every interaction (timestamp, user, content, importance score). Add semantic search over episodes using ChromaDB.    | Lab 6.1: Build a memory-augmented chatbot. On session start, retrieve the 5 most relevant past interactions using recency+relevance blending. Show that the agent references past context correctly.              |
| <b>Tue</b> | <b>Semantic Memory &amp; Profiles</b> | Build a semantic memory store that extracts and stores facts about users from conversations (name, preferences, goals, constraints). Use an LLM to extract facts into structured Pydantic models, then embed and store. | Build a user profile builder: after 10 interactions, agent synthesises a structured profile from episodic memory. Profile updates automatically. Show that agent behaviour changes based on the profile.          |
| <b>Wed</b> | <b>Knowledge Graphs</b>               | Study graph theory basics: nodes, edges, directed vs undirected, weighted edges. Install NetworkX. Build a knowledge graph from 20 Wikipedia paragraphs: extract entities, classify relationships, add to graph.        | Lab 6.2: Build a knowledge graph query agent. User asks a natural language question → agent translates to graph traversal → returns answer with the reasoning path. Visualise the graph with pyvis or matplotlib. |
| <b>Thu</b> | <b>GraphRAG &amp; Consolidation</b>   | Study GraphRAG pattern: run vector retrieval AND graph traversal in parallel, merge context, deduplicate. Compare answer quality on multi-hop questions vs pure vector RAG.                                             | Build memory consolidation: when episode count exceeds 50, agent summarises oldest 25 episodes into a compressed memory block. Implement importance-based                                                         |

|            |                               |                                                                                                                                                                                       |                                                                                                                                                                                 |
|------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            |                               |                                                                                                                                                                                       | forgetting: low-importance episodes decay over time.                                                                                                                            |
| <b>Fri</b> | <b>Full Stack Integration</b> | Integrate all four memory types into one agent. Add a memory inspector UI (Streamlit): show episodic log, semantic profile, knowledge graph, and procedural corrections side by side. | Standup: demo memory persistence across sessions. Show agent improving over 5+ interactions. Architecture review: where does each memory type live? What are the failure modes? |

# Recommended Resources

The MemGPT and cognitive architecture papers are required reading before Wednesday. The GraphRAG blog is essential before Thursday.

| Type  | Resource                                                        | Why It Matters                                                                   |
|-------|-----------------------------------------------------------------|----------------------------------------------------------------------------------|
| Paper | MemGPT: Towards LLMs as Operating Systems (2023)                | Foundational defines the layered memory model for LLM agents                     |
| Paper | Cognitive Architectures for Language Agents (CoALA, 2023)       | Comprehensive survey of memory types and agent cognition                         |
| Paper | From RAG to GraphRAG: Knowledge Graph-Enhanced Retrieval (2024) | Technical foundation for combining vector + graph retrieval                      |
| Docs  | Mem0 Documentation (mem0.ai)                                    | Production memory framework study its architecture even if not using it directly |
| Docs  | NetworkX Documentation                                          | Graph construction, traversal, and analysis in Python                            |
| Docs  | ChromaDB Persistence Documentation                              | Long-term semantic memory storage                                                |
| Video | Building Memory Systems for AI Agents (LangChain YouTube)       | Practical implementation walkthrough                                             |
| Blog  | Microsoft GraphRAG: Unlocking LLM Discovery (2024)              | Microsoft's production GraphRAG system explained                                 |
| Blog  | LlamaIndex Building Stateful Agents with Memory (2024)          | Practical patterns for agent memory                                              |
| Repo  | Mem0 GitHub Repository                                          | Reference implementation of a production memory system                           |

|      |                                                                |                                      |
|------|----------------------------------------------------------------|--------------------------------------|
| Blog | Knowledge Graphs for RAG: A Practical Engineering Guide (2025) | Engineering guide with code examples |
|------|----------------------------------------------------------------|--------------------------------------|



# Hands-on Labs

Three labs, increasing in complexity. Lab 6.3 is the most technically demanding lab in the programme so far – start it on Thursday.

## Lab 6.1 Memory-Augmented Chatbot

Build a CLI chatbot backed by two memory stores: a SQLite episodic log (raw interaction history with timestamps) and a ChromaDB semantic index (embedded summaries of past sessions). On every new session, the agent queries both stores and injects the 5 most relevant past memories into context before responding. Demonstrate that the agent correctly references something said in a previous session without it being in the current context window.

**Validation** Test by running 3 separate sessions. In session 3, ask about something only mentioned in session 1. The agent must answer correctly using retrieved memory.

## Lab 6.2 Knowledge Graph Query Agent

Build a knowledge extraction and query pipeline. Feed 20 text paragraphs (Wikipedia articles on a domain of your choice) to an LLM that extracts entities (people, companies, places, concepts) and relationships (works\_at, founded\_by, located\_in, part\_of). Store the graph in NetworkX. Build a query agent that converts natural language questions into graph traversals and answers multi-hop questions that require crossing at least two edges to answer. Visualise the final graph with pyvis (HTML output).

**Validation** Test with 5 multi-hop questions. At least 4 must be answered correctly by traversing the graph. A pure keyword search of the original text must fail on at least 2 of the same questions.

## Lab 6.3 GraphRAG: Vector + Graph Hybrid Retrieval

Build a GraphRAG pipeline: given a question, run vector retrieval (ChromaDB top-5) and graph traversal (NetworkX neighbourhood expansion) in parallel. Merge the contexts, deduplicate, and pass to an LLM for generation. Evaluate on 15 questions split into three categories: factual (vector wins), relational (graph wins), complex (hybrid wins). Report results in a structured comparison table. Implement automatic

query routing: a classifier decides whether to use vector-only, graph-only, or hybrid based on the question type.

**Validation** Hybrid must outperform both vector-only and graph-only on complex questions. Query router must correctly classify at least 12 of 15 test questions.

# Weekly Assessment

Answer all six questions in writing before Friday's standup. Be precise – vague answers will be challenged.

| Type       | Question                                                                                                                                                                                                                                                                        |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Conceptual | Explain the difference between episodic and semantic memory in an AI agent. Give a concrete example of each that matters in a production system.                                                                                                                                |
| Conceptual | What is memory consolidation? Why is it necessary, and what are the risks of getting it wrong?                                                                                                                                                                                  |
| Conceptual | When does a knowledge graph outperform vector retrieval, and when does it fail? What types of questions expose each weakness?                                                                                                                                                   |
| Technical  | Design the SQLite schema for an episodic memory store that supports recency weighting, importance scoring, and per-user isolation.                                                                                                                                              |
| Technical  | Explain how you would implement importance-based memory forgetting. What signals determine importance, and how do they decay over time?                                                                                                                                         |
| Design     | You are building a personal AI assistant that must remember users across months of interactions without the context window growing unboundedly. Design the full memory architecture: what gets stored where, when memories are compressed, and what gets permanently forgotten. |

# Weekly Standup Requirements

Prepare each item before Friday. The memory demonstration must show genuine cross-session persistence not in-context memory.

| Item                 | Expectation                                                                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cross-Session Demo   | Show an agent remembering something from a previous session that is not in the current context window. Must be verifiable show the database state before and after. |
| Memory Inspector     | Show a UI or terminal view of all four memory stores side by side: episodic log, semantic profile, knowledge graph, procedural corrections.                         |
| Architecture Review  | Draw the full memory architecture: every store, every read/write path, and every trigger for memory consolidation.                                                  |
| Knowledge Graph Demo | Run a live multi-hop query on your knowledge graph. Show the traversal path, not just the answer.                                                                   |
| Improvement Evidence | Show a before/after comparison: agent response quality after 0 interactions vs after 10 interactions with memory enabled.                                           |
| Reflection           | What was the hardest memory engineering decision you made this week? What would you change with another day?                                                        |

# Week 6 Projects

Memory is where AI systems become genuinely personal and genuinely useful over time. This week's projects are designed to demonstrate exactly that systems that get better with every interaction, not systems that reset.

**Portfolio Standard** Every project must include a demonstrable before/after comparison: agent behaviour with zero memory vs agent behaviour after N interactions. A project that cannot prove it learns is not complete.

## Category 1 Projects Choose One

Intermediate projects should take 7–10 hours. All three require genuine cross-session memory and a demonstration of measurable improvement.

### Project 6-I-A: Long-Term Personal Research Assistant

| Long-Term Personal Research Assistant |                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>                     | Intermediate                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Overview</b>                       | Build a research assistant that genuinely improves with use. After each session it extracts what the user cares about, what questions they asked, what answers they liked, and builds a structured user model. Future sessions are personalised: topics the user has explored before are summarised rather than re-explained, and the agent proactively connects new information to past research. |
| <b>Problem Solved</b>                 | Every AI assistant today starts from zero. A research tool that remembers what you already know, what you care about, and how you like information presented is categorically more useful and almost no one has built it properly.                                                                                                                                                                 |
| <b>System Architecture</b>            | Session Input → Interaction Logger (SQLite episodic store) → Fact Extractor Agent (Pydantic user model) → Semantic Profile Store (ChromaDB) → Session Initialiser (retrieves relevant memories + profile at session start) → Research Agent (Groq) → Post-Session Memory Writer → Streamlit UI with session history panel                                                                          |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Memory Design</b>           | Episodic Store (SQLite): raw interaction log with timestamp, session_id, content, and importance_score. Semantic Store (ChromaDB): embedded user facts, preferences, and research interests tagged with recency and domain. User Profile (SQLite JSON field): structured aggregate known topics, preferred depth, communication style, open questions. Memory Writer runs after every session and updates all three stores atomically. |
| <b>Technical Skills</b>        | Multi-layer memory architecture, LLM-driven fact extraction, Pydantic user models, ChromaDB persistence, SQLite design, Streamlit multi-page UI, session management                                                                                                                                                                                                                                                                    |
| <b>Portfolio Deliverables</b>  | GitHub repo with system design README, Streamlit app with 3-panel UI (current session / memory inspector / profile viewer), 5-session demo transcript showing measurable personalisation over time, memory schema documented, before/after quality comparison, architecture diagram                                                                                                                                                    |
| <b>What Makes It Stand Out</b> | The user profile viewer showing the agent's model of who you are is a striking demo moment. The before/after comparison (session 1 vs session 5) makes the improvement concrete and visible. Most AI assistants reset; this one grows.                                                                                                                                                                                                 |
| <b>Evaluation Criteria</b>     | Agent correctly references past research in at least 3 of 5 test sessions. User profile updates after every session with at least one new fact. Personalisation is demonstrably different between session 1 and session 5. Streamlit UI shows memory state in real time. All three memory stores persist across app restarts.                                                                                                          |

## Project 6-I-B: Domain Knowledge Graph Builder & Explorer

| Domain Knowledge Graph Builder & Explorer |                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>                         | Intermediate                                                                                                                                                                                                                                                                                                                                                                              |
| <b>Overview</b>                           | Build an interactive knowledge graph platform for a specific domain (choose one: AI research papers, company ecosystems, historical events, biomedical concepts). The system automatically extracts entities and relationships from new documents, adds them to a persistent graph, and lets users explore the graph through natural language questions that require multi-hop reasoning. |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem Solved</b>          | Search engines find documents. Knowledge graphs find relationships. A platform that automatically builds a structured world model from raw documents and lets you query it conversationally is a fundamentally different kind of knowledge tool.                                                                                                                                                                                                        |
| <b>System Architecture</b>     | Document Input (PDF/text) → Entity-Relationship Extractor Agent (LLM + structured Pydantic schema) → Deduplication Layer (merge existing entities) → NetworkX Graph (persisted as JSON) → Query Classifier (factual / relational / complex) → [Vector Retrieval (ChromaDB)   Graph Traversal (NetworkX)] → GraphRAG Merger → Answer Generator → Interactive pyvis Graph UI (HTML) → Streamlit Control Panel                                             |
| <b>Memory Design</b>           | Knowledge Graph (NetworkX, serialised as JSON): entities as nodes (with type, description, source_doc attributes), relationships as directed edges (with type, confidence, source_doc). Persists to disk and reloads on startup. Vector Index (ChromaDB): entity descriptions embedded for fuzzy entity matching and seeding graph traversal start nodes. Extraction log (SQLite): tracks which documents have been processed to prevent re-extraction. |
| <b>Technical Skills</b>        | Entity and relationship extraction, graph deduplication, NetworkX graph algorithms, GraphRAG, pyvis interactive visualisation, ChromaDB for entity matching, multi-hop query decomposition                                                                                                                                                                                                                                                              |
| <b>Portfolio Deliverables</b>  | GitHub repo, Streamlit app with document ingestion + graph explorer + Q&A interface, interactive pyvis graph (committed as HTML), corpus of 30+ ingested documents, 15 multi-hop Q&A pairs demonstrating graph reasoning, before/after comparison (vector-only vs GraphRAG), architecture diagram, recorded 3-minute demo                                                                                                                               |
| <b>What Makes It Stand Out</b> | An interactive, explorable knowledge graph built automatically from documents is visually compelling and technically impressive. The multi-hop Q&A demonstration showing reasoning paths across the graph is the kind of demo that stops an interviewer mid-sentence. Very few interns build this.                                                                                                                                                      |

|                            |                                                                                                                                                                                                                                                                                                                                              |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Evaluation Criteria</b> | Graph ingests 30+ documents with accurate entity and relationship extraction. Multi-hop queries answered correctly across at least 10 of 15 test questions. pyvis graph renders and is explorable. GraphRAG outperforms vector-only on at least 4 of 5 relational test questions. New documents added at runtime update the graph correctly. |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Project 6-I-C: Procedural Memory & Self-Improving Agent

| Procedural Memory & Self-Improving Agent |                                                                                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>                        | Intermediate                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Overview</b>                          | Build an agent that learns from its own mistakes. When a user corrects an answer, the agent extracts the correction as a procedural memory (a rule), stores it, and ensures the same mistake never happens again. Over 20 interactions, the agent's error rate measurably decreases. The system tracks its own performance and plots a learning curve.                                             |
| <b>Problem Solved</b>                    | Current AI systems make the same mistakes repeatedly because they have no procedural memory. An agent that extracts rules from corrections and applies them prospectively is a fundamentally different class of system and the architecture to build it is non-trivial.                                                                                                                            |
| <b>System Architecture</b>               | User Input → Response Generator (Groq) → User Feedback Handler → [if correction: Correction Extractor Agent → Rule Store (SQLite)] → Rule Retriever (retrieves relevant rules at generation time) → Rule-Augmented Prompt Builder → Improved Response Generator → Performance Tracker (SQLite) → Learning Curve Visualiser (matplotlib) → Streamlit Dashboard                                      |
| <b>Memory Design</b>                     | Procedural Store (SQLite): correction rules with fields: original_mistake, correction, rule_text, domain, confidence, application_count, last_applied. Rules retrieved by semantic similarity to current query (embedded with sentence-transformers). Application log tracks every time a rule fires. Rule consolidation: after 5 identical corrections, merge into a single high-confidence rule. |
| <b>Technical Skills</b>                  | Procedural memory design, LLM-driven rule extraction, rule retrieval by semantic similarity, dynamic prompt construction, performance tracking, matplotlib learning curves, Streamlit dashboard                                                                                                                                                                                                    |



|                                |                                                                                                                                                                                                                                                                                                                 |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Portfolio Deliverables</b>  | GitHub repo, Streamlit dashboard with live rule book, correction interface, and learning curve chart, 20-interaction demonstration dataset showing error rate improvement, rule extraction quality analysis, comparison of agent behaviour before/after 20 corrections, architecture diagram                    |
| <b>What Makes It Stand Out</b> | A system that plots its own learning curve is a remarkable demo artifact. The rule book – a visible, inspectable list of things the agent has learned not to do – makes procedural memory tangible. Error rate charts are evidence of learning that interviewers remember.                                      |
| <b>Evaluation Criteria</b>     | Agent correctly applies learned rules in at least 80% of cases where the rule is relevant. Error rate decreases measurably between interactions 1–5 and interactions 16–20. Rule book is inspectable via UI. Correction extraction produces coherent, applicable rules. Learning curve chart committed to repo. |

## Category 2 Projects Choose One

Production projects should take 10–15 hours. These are complete platforms deployed, documented, and compelling enough to lead a portfolio.

**Hiring Signal** A production memory system that demonstrably improves over time is one of the most distinctive portfolio pieces an AI engineer can show in 2026. Almost no one builds this correctly.

### Project 6-P-A: Enterprise AI Memory Platform

| Enterprise AI Memory Platform |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>             | High                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Overview</b>               | Build a production-grade memory platform as a standalone service. Any AI agent can connect to it via REST API to read and write all four memory types. Supports multi-tenant isolation (each user/organisation has a separate memory namespace), cross-session persistence, knowledge graph construction, and a rich admin dashboard showing memory state across all tenants.                                                                                                         |
| <b>Problem Solved</b>         | Memory is currently rebuilt from scratch in every AI application. A memory-as-a-service platform means any agent regardless of how it is built can have persistent, structured, queryable memory without reimplementing the infrastructure each time. This is what Mem0 is building; you are building your own, deeper version.                                                                                                                                                       |
| <b>System Architecture</b>    | FastAPI Memory Service → [Episodic API (SQLite, per-tenant tables), Semantic API (ChromaDB, per-tenant collections), Procedural API (SQLite rule store), Knowledge Graph API (NetworkX, per-tenant JSON)] → Memory Router (decides which store to read/write based on memory type) → Consolidation Worker (async background task, runs every 100 episodes) → Admin Dashboard (Streamlit, shows all tenants and memory state) → Docker Compose (FastAPI + Streamlit + ChromaDB server) |
| <b>Memory Design</b>          | Episodic Store: SQLite with tenant_id, session_id, timestamp, content, embedding_id, importance_score. Semantic Store: ChromaDB with tenant-namespaced collections, metadata filtering. Procedural Store: SQLite rule table with tenant_id, domain, rule_text, confidence, application_count. Knowledge Graph: per-tenant NetworkX graphs serialised as JSON to disk, loaded into memory on first access. Consolidation Worker: async task that summarises old episodes,              |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                | promotes high-confidence rules, and prunes low-importance memories on a configurable schedule.                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Technical Skills</b>        | Memory-as-a-service architecture, multi-tenant isolation, async background workers, FastAPI advanced patterns, ChromaDB collections, SQLite multi-tenant design, Streamlit admin UI, Docker Compose orchestration, REST API design                                                                                                                                                                                                                                                             |
| <b>Portfolio Deliverables</b>  | GitHub repo (public, portfolio-ready), full system design README with architecture diagram, Docker Compose one-command deployment, FastAPI with complete OpenAPI documentation, Streamlit admin dashboard showing live memory state for 3 demo tenants, integration example showing an external LangChain agent using the memory API, RAGAS-style evaluation of retrieval quality across all memory types, recorded 5-minute demo video, blog post explaining the memory platform architecture |
| <b>What Makes It Stand Out</b> | This is a real product. Mem0 raised \$10M building exactly this. Your open-source version deployed, documented, with a working agent integration demonstrates product-level engineering thinking. The admin dashboard alone is worth showing to any hiring manager.                                                                                                                                                                                                                            |
| <b>Evaluation Criteria</b>     | All four memory types accessible via REST API. Multi-tenant isolation verified (tenant A cannot read tenant B's memories). Consolidation worker runs correctly and reduces episode count. Admin dashboard shows accurate memory state. Docker Compose starts full platform in one command. External agent integration example included and functional. OpenAPI docs complete.                                                                                                                  |

## Project 6-P-B: GraphRAG Knowledge Intelligence System

| GraphRAG Knowledge Intelligence System |                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>                      | High                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Overview</b>                        | Build a production GraphRAG system that outperforms standard RAG on complex, multi-hop questions. The system ingests a large corpus (50+ documents), builds both a vector index and a knowledge graph simultaneously, implements intelligent query routing, and proves its superiority with a rigorous evaluation study including a blog post publishing the results. |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem Solved</b>          | Standard RAG retrieves chunks. GraphRAG retrieves reasoning chains. For questions that require understanding relationships between entities across documents 'What did Company A and Company B have in common that Company C lacked?' vector retrieval consistently fails. This project proves you can build the system that solves it.                                                                                                                                                      |
| <b>System Architecture</b>     | Document Corpus → Dual Indexer: [ChromaDB Pipeline (chunk → embed → store) + Graph Pipeline (entity extract → relationship extract → NetworkX or Neo4j → persist)] → Query Analyser (classifies: factual / relational / multi-hop) → Smart Router → [Vector Retriever   Graph Traverser   Hybrid Merger] → Context Ranker and Deduplicator → LLM Generator (Groq) → Answer + Evidence Chain → FastAPI endpoint → Streamlit Research UI → Evaluation Framework (RAGAS + custom graph metrics) |
| <b>Memory Design</b>           | Knowledge Graph: entities as nodes (type, canonical_name, aliases, source_documents), relationships as directed edges (type, evidence_text, confidence, bidirectional flag). Stored in NetworkX (serialised JSON) or Neo4j Aura (free tier). Graph is updated incrementally new documents add nodes and edges without rebuilding. Entity resolution: LLM-based canonical name matching prevents duplicate nodes for the same entity.                                                         |
| <b>Technical Skills</b>        | Dual indexing pipeline, entity resolution, Neo4j Cypher or NetworkX algorithms, intelligent query routing, context deduplication, GraphRAG evaluation, RAGAS, statistical comparison, FastAPI, Streamlit, Docker                                                                                                                                                                                                                                                                             |
| <b>Portfolio Deliverables</b>  | GitHub repo, 50-document ingested corpus (domain of your choice), FastAPI with OpenAPI docs, Streamlit research UI with query mode selector, rigorous evaluation study comparing vector-only vs graph-only vs hybrid on 30 questions (10 per category), statistical analysis (t-test or equivalent), HTML evaluation report, architecture diagram, recorded 5-minute demo, published blog post with evaluation results (dev.to or Hashnode)                                                  |
| <b>What Makes It Stand Out</b> | Publishing an evaluation study with real numbers is rare at any level. A chart showing 'GraphRAG outperforms RAG by 34% on multi-hop questions' is a concrete, shareable finding. Hiring managers remember papers and blog posts; they forget projects that are just code.                                                                                                                                                                                                                   |
| <b>Evaluation Criteria</b>     | 50+ documents ingested with accurate dual indexing. Query routing classifies correctly on at least 25 of 30 test questions. GraphRAG outperforms vector-only by a statistically significant margin on                                                                                                                                                                                                                                                                                        |

|  |                                                                                                                                                                                |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | multi-hop questions. RAGAS scores reported for all three retrieval modes. Evaluation report committed to repo. Blog post live and linked from README. Docker deployment works. |
|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Project 6-P-C: Autonomous Learning & Adaptation Platform

| Autonomous Learning & Adaptation Platform |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Difficulty</b>                         | High                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Overview</b>                           | Build a platform where an AI agent autonomously builds a structured model of a domain (knowledge graph), tracks what it knows and does not know (semantic memory), learns from corrections (procedural memory), and adapts its communication style per user (episodic memory). The system is designed around one guiding principle: every interaction makes the agent demonstrably better.                                                                                                                                                                                                                             |
| <b>Problem Solved</b>                     | The gap between a demo AI and a production AI is not capability it is memory. A system that gets measurably smarter with use, that knows what it does not know, and that adapts its behaviour per user is a production-grade AI. Most AI engineers cannot build this. You will.                                                                                                                                                                                                                                                                                                                                        |
| <b>System Architecture</b>                | User Interaction → Session Memory Manager → Episodic Logger (SQLite per user) → Knowledge Gap Detector Agent (identifies unanswered questions or low-confidence answers) → Active Learning Queue (topics for self-study, priority-ranked) → Knowledge Builder Agent (researches gap topics, adds to NetworkX knowledge graph) → Correction Handler → Procedural Memory Updater → User Profile Builder → Adaptive Response Generator (injects all memory types into context) → Performance Tracker → FastAPI Multi-Tenant API → Streamlit Dashboard (per-user memory state, knowledge graph, learning curve, gap queue) |
| <b>Memory Design</b>                      | Four fully integrated memory types: Episodic (SQLite, per-user interaction log with importance scoring and recency decay), Semantic (ChromaDB per-user collection + structured user profile in SQLite JSON), Procedural (SQLite rule store with domain classification and confidence scores), Knowledge Graph (NetworkX per-domain graph, shared across users but queryable per-user context). Knowledge Gap Queue (SQLite priority queue of unresolved questions). Active Learning Log (SQLite, tracks what the agent researched autonomously and what it learned).                                                   |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Technical Skills</b>        | Full memory stack integration, knowledge gap detection, active learning architecture, autonomous knowledge graph expansion, per-user adaptation, multi-tenant FastAPI, Streamlit advanced UI, Docker Compose, comprehensive evaluation                                                                                                                                                                                                                                            |
| <b>Portfolio Deliverables</b>  | GitHub repo, Docker Compose deployment, FastAPI with OpenAPI docs, Streamlit multi-panel dashboard (memory inspector, knowledge graph explorer, learning curve, gap queue, user profile), 5-user simulation demonstrating per-user adaptation, knowledge graph growth over 50 interactions visualised, autonomous learning demonstration (3 gap topics researched and added to graph), performance metrics showing improvement over time, recorded 5-minute demo video, blog post |
| <b>What Makes It Stand Out</b> | The knowledge gap detector – an agent that knows what it does not know and autonomously fills those gaps – is a concept from cognitive science applied to AI engineering. A dashboard showing a knowledge graph growing in real time as the system learns is unlike any other portfolio project. This is the kind of system that generates interview invitations.                                                                                                                 |
| <b>Evaluation Criteria</b>     | All four memory types integrated and persist across app restarts. Knowledge gap detector correctly identifies at least 5 genuine knowledge gaps in a 20-interaction test. Knowledge Builder Agent adds at least 3 researched topics to the graph autonomously. Per-user adaptation demonstrably different between user 1 and user 5 after 10 interactions each. Performance metrics show improvement. Docker Compose starts full platform. Dashboard functional with all panels.  |

# Week 6 Completion Checklist

Every item must be confirmed before Friday's standup. Cross-session memory persistence is verified live – there is no way to fake it.

|                          | Checklist Item                                                                                       |
|--------------------------|------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> | All 5 daily learning sessions completed (Mon–Fri)                                                    |
| <input type="checkbox"/> | Lab 6.1 Memory-augmented chatbot demonstrating cross-session recall across 3 separate sessions       |
| <input type="checkbox"/> | Lab 6.2 Knowledge graph built from 20+ documents with working multi-hop query agent                  |
| <input type="checkbox"/> | Lab 6.3 GraphRAG pipeline built with query router and comparison evaluation completed                |
| <input type="checkbox"/> | Weekly Assessment – all 6 questions answered in writing                                              |
| <input type="checkbox"/> | One Intermediate project chosen, built, and pushed to GitHub with full README                        |
| <input type="checkbox"/> | One Production project chosen, built, and pushed to GitHub with system design README                 |
| <input type="checkbox"/> | Both projects include a demonstrable before/after comparison (0 memory vs N interactions)            |
| <input type="checkbox"/> | Architecture diagrams showing all memory stores and their read/write paths committed to repo         |
| <input type="checkbox"/> | Demo rehearsed – must show cross-session memory, knowledge graph traversal, and improvement evidence |